* How to solve a particular problem?

① Think - 10 min
② Sol^n found → code it, if not → see hints → code it } using Brute force approach
③ find optimized sol^n
④ find more optimized sol^n

In notes for every approach ⇒ code along with algorithm + dry run using short example + T.C & S.C + Imp point (which you learnt new ~~during the~~ while solving problem.)
first write down which datastructure & pattern you used?

Notes: ① Topic / datastructure / algorithm / Patterns of solving probl. related to that datastructure.
② Main Goal: (i.e. what you're gonna get after solving that problem using it) or short intro of topic or def^n.
(why are you using this datastructure/algorithm/pattern)
③ algorithm (what you're gonna do in it).
④ Step by step implementation using 1 hard example (How you're doing it.)
⑤ Code:-
⑥ Imp points to remember (new or hard to remember.)

Lec. 1:- Git & Github → read Basic git commands with examples & Git cheatsheet from gfg.

locally → local machine (ac. on local machine)
remotely → github online ac.

① mkdir Project → make folder to save all files of your <Project>.
② cd Project → Go in Project folder.
③ git init → intialize empty local git repo. in <Project> folder itself (where all changes / history of your project (done in <Project> folder) will be saved.
④ git config --global user.name "Gaurj" → author of .git folder (required to save hist. of your proj / to save which person made that change / commit) to name the person who is making changes in .git folder
git config --list → to show author's name in git log
② git status → to track all changes made in <Project> folder
to watch what is their in local machines git repo. (history of your proj) which is present in <Project> folder.
(i.e. ~~history of your proj~~) it will ~~contain~~ show (tracked / untracked files) → i.e. modified files / files not in .git folder but present in project folder

⑤ untracked files → files present in <Project> folder but not added in .git folder (i.e. this file is made in <Project> folder → This thing/history of <Project> folder is not added in .git folder).

⑥ git restore --staged names.txt → to unstage change made in names.txt file before/without committing it.
(or to remove changes made in names.txt file from staging area or .git (local) folder)
& not from working directory (i.e. local machine)
↓
you can add modified changes again just by doing git add. (which you had unstaged)

⑦ 3 insertions(+) → i.e. added 3 lines in a file which got changed → (can say approximately / maybe 3 lines added) or 3 files added (another of)

⑧ git status → shows staging area (~~to what's inside~~

⑨ git log → shows committed changes history

⑩ rm -rf names.txt → to delete a file. from local machine folder & not .git folder. working directory as well
→ to save this change in .git folder
→ use git add . command.
(file deleted.)

⑪ terminal type → xterm

⑫ git reset <~~id~~ commit ID> → to remove all commits above this <~~id~~> & files/commits which you have removed using this command will go in the ~~staging area~~ unstaging area / will be unstaged.
git reset --hard <~~id~~>
(changes will be removed from working dir. as well.)
but changes won't removed from working dir.

⑬ git stash :- to remove all staged / modified changes ~~before~~ done after last commit or to remove all staged modified ~~sta~~ changes before committing. (but not untracked files) from .git folder ~~as well~~ & working dir. as well. ~~from~~ to stashing area

⑭ git stash pop :- to stage all changes present in stashing area. ~~in~~ ~~of~~

⑮ git stash clear :- to remove/delete/clear all ~~stash~~ changes present in stash area. permanently.

(16) git remote add origin <https URL of your own repo. present in your own ac.> → to connect remote git repository online to your local project's folder / local project

(17) git remote -v → shows all URL's attached with current directory/ folder (i.e Project Repo)

(18) git push origin ~~master~~ →
- without commits you can't push any branch to your remote repo. (present origin)
→ before pushing any local branch to remote repo → first fetch all the changes present on github repo. online (from origin as well as upstream → main branch) → given on next page.

(18) git pull origin main --rebase → to pull all changes present in github repo. online (present in your own ac & currently attached to your .git folder)

&

her assumption of this line)

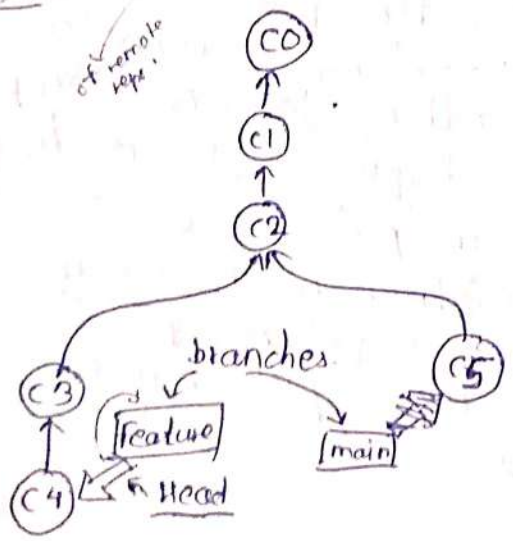
(19) git push origin master → to push current local branch to remote repo. (~~on~~ origin)

local branch which you've created in your own github a/c online → can push changes to cloned repo as well

branch of remote repo. on which we have to push current branch changes

* New feature = New branch.
→ Never commit on main branch (or default branch)
→ Branches :-

dir



Head can only pt to one branch at a time (here, feature)

→ origin (shortcut name for your Github repo's URL)
either cloned or your own
but should be in your own ac

(20) git clone <URL of fork> → to clone ~~exis~~ fork (copy of existing proj. in your own ac.) on your local machine / current ~~local~~ ~~github~~-ac. directory.
origin URL

(21) git remote add upstream <URL> → to connect upstream URL to your current directory
upstream URL

~~* Pull request~~ :→ ~~Even if you have done changes in main branch but it was new change. → & new change = new branch, so after you're done your change on main branch, add that change in new branch & commit on new branch (as we discussed never commit on main branch) → how? see below.~~

~~git c~~
→ new feature = new branch = new pull request

* Pull request :- Request done by you to main organization (their) to merge your change into main project from Local repo → remote repo → main projects main branch.
Journey of pull request.

→ Process to create Pull request :- ① first push changes from Local repo → remote repo
② then click on "compare & Pull request" or "create pull request" in "pull request section" of your remote repo
③ then main organization will get notification as merge pull request.

(Force pushing overwrites the remote branch history, deleting any commits on the remote that are not in your local branch, can cause other collaborators work, so use it only on private branches with extreme caution)

Rule:- One branch = one pull request.

(22) git push origin kunal -f → force push changes (kunal branch) from local repo to remote repo (which will reflect in main organization as well. so her (kunal branch not new branch)

(after you've create it from main remote repo by clicking on create pull req)

(22.2) git checkout main → then do next

(23) git pull upstream main → it pulls/fetches changes made in main branch of upstream in local main branch

-prune → deleted ones as well

(to update your main branch of fork with main project's [as well] main branch by pulling their changes to fork)

you can do

(24) git push origin main → to push whatever changes you've fetched from main project's main branch to remote repositories main branch /origin URL.

~~seper~~

* Squashing Commits:

→ before creating new branch → make sure your main branch is upto date

→ git reset <#> → unstage all commits above this hash

→ git add . → put them back in staging. all at (i.e. Git stages current state of your working directory) it doesn't revert commits but reverts all changes (from commits you reset a way) will be squashed into 1 single commit not separate commits.

→ git commit -m "Redo commit" → so all reverted ↓ unstaged changes & files

* Squashing commits:-

- git branch temp
- git checkout temp
- gitouch 1
- git add .
- git commit -m "1"
- touch 2
- git add .
- git commit -m "2"
- git rebase -i <#> ⇒ commit ID → ~~above which~~ commits above which <#> we have to either pick/squash.

→ pick 1
s 1 → it will merge whatever commit is listed as 's' into its above commit

which is listed as picked one

which you've created directly from Local repo

→ main organization merges your pull request of their branch into main branch

type i to go in insert mode
→ ~~escape + :x~~ ↓ → to create one message for all merged commits.
or
:wq

→ escape + :x + enter → to exit from interactive environment & now you can push these changes to your remote repo. & then to main organizations main branch
or
:wq

* Merge conflicts:-

occurs when 2 branches modify the same part of a file differently & Git can't automerge

Fix:- edit file → resolve conflict → remove markers → git add <file> → git commit -m "resolved conflict".

JMP Points:

* only write what you don't know/tend to forget → Nothing else!!
* Please be very limited with your resources → keep notes minimized/limited as possible
* try to adapt as soon as possible → as tech is changing very fast (but basics will remain same e.g. DSA) ↓ focus → understanding & not mugging (skip things which you need to mug up → as you'll get used to required things when you start coding everyday.
* learn by doing rather than writing & reading everyday & mugging ↓ instead, do it along with lectures & remember for lifetime.
* When you don't feel like studying → Just see salary of engineers at levels.fyi
* Don't do stuff like learning extra langs as it doesn't add any value & not required to get placed.